

Runview — Product & Technical Documentation

Self-hosted test-intelligence platform: a results portal, an AI test-authoring agent, and a full test-management workspace, in one app.

Stack: FastAPI (Python 3.12) + React 19 / Vite SPA + PostgreSQL 17 (pgvector) · **Deploy:** Docker Compose · **Migrations:** Alembic head **0024**

Latest additions — an end-to-end **in-app automation** subsystem: generate & run Playwright tests for a case from the UI, with a choice of execution backend (an **in-built Docker runner** or **CI** — Bitbucket Pipelines / AWS CodeBuild / webhook), a live **automation queue**, an approval-gated **PR flow**, multi-workspace support, and an **inline-checklist cycle runner**. Plus an in-app **interactive product tour**. See §4A and §6.

1. What Runview is

Runview began as a lightweight CI test-results portal ("ReportPortal" internally) and has grown into a three-pillar product. All three pillars share one FastAPI backend, one Postgres database, one React SPA, and a common auth/RBAC and workspace model.

Pillar	Purpose	Entry point
Results Portal (core)	Ingest CI/automation results, dashboards, run/test history, flaky analysis, deployment monitor, PDF reports	Home , Results
SDET Agent	AI (AWS Bedrock) impact analysis on Jira stories + code → proposes/updates test cases, human approval, push to TestSigma	Agent
Test Library (TestManagement)	Full manual/automated test-case management: repository, suites, plans, cycles, execution, requirements, defects, reviews	Test Library

Core concepts

- **Workspace** = a RunView **Project** (slug, e.g. `arm` , `guard`). Everything (runs, cases, agent data) is scoped to a workspace. Switching workspace re-scopes the whole app.
- **Roles:** `viewer` (read), `editor` (create/edit), `superadmin` (admin/settings/destructive ops).
- **Per-workspace module enablement:** SDET Agent & Test Library activate only for workspaces registered in `sdet_workspaces` ; others show a friendly "not enabled" state.

2. Pillar 1 — Results Portal (core)

2.1 Ingestion (how results get in)

CI systems push results via an **API key** (`X-API-Key` header), no user login needed:

- **JSON batch** — `POST /api/projects/{ref}/runs`
- **JUnit XML** — multipart upload (XXE-hardened parser)
- **Mabl JSON** — multipart upload
- **Live streaming** — `runs/live/start` → `.../events` → `.../finish` for real-time run building
- **Media** — separate attachment upload (screenshots/video), 25 MB limit, classified by kind

Hierarchy: **Project** → **Run** → **TestResult** → (**TestStep**, **TestLog**, **Attachment**).

2.2 Features

- **Dashboard** — configurable widgets: pass rate, failures, live runs, tests executed, velocity, execution time, trends, top flaky tests; widget add/delete/resize.
- **Runs & Run Detail** — filterable run list; run detail with test tree grouped by module/class, inline logs, step-by-step results, screenshots/video, and inline per-test history.
- **Tests catalog & history** — per-test-case history across runs; flaky-test detection.
- **Quality & Monitor** — quality metrics and a live deployment/environment health monitor, each with on-demand AI analysis.
- **PDF report** — Extent-style printable run report.
- **Retention** — automatic daily purge of runs older than the configured window (cascades to tests/logs/attachments + disk files).

2.3 Core data model

`User`, `Project`, `ApiKey`, `Run`, `TestResult`, `TestStep`, `TestLog`, `Attachment` (media), `AppSetting` (encrypted config), `DashboardWidget`, `InstanceMapping`, `AiUsage` (token/cost ledger).

3. Pillar 2 — SDET Agent (AI test authoring)

An asynchronous, human-in-the-loop agent that turns a Jira story (+ code/PR context) into proposed test cases.

3.1 Pipeline (end to end)

1. **Trigger** — `POST /api/sdet-agent/runs {workspace_id, story_key, tester_prompt?}` → queued (DB-backed queue, no Redis/Celery).
2. **Context pull (Phase 4)** — fetch the Jira story (summary, components, links, fix versions), PR/dev-status info (repos, changed files via GitHub), optional **Rovo** AI enrichment.
3. **Retrieval (Phase 5a)** — build a query embedding (Amazon Titan, 1024-dim) and pull **7 lenses** via pgvector cosine search: candidate TestSigma cases, KB pages, code symbols, historical defects,

Salesforce metadata, integration touchpoints, user context docs.

4. **Analysis (Phase 5b)** — AWS **Bedrock (Claude)** produces decisions classified **REUSE / UPDATE / NEW**, each with reasoning, lens evidence, confidence, and a full proposed test case (preconditions + steps). A **60% overlap gate** biases toward reuse; an optional **performance-impact** assessment runs in parallel.
5. **Review & approval** — editors edit; superadmins approve/reject. A **similarity guard (85%)** blocks near-duplicate NEW cases.
6. **Push to TestSigma (reversible)** — NEW → create, UPDATE → update, REUSE → no-op. Before/after snapshots stored so any push can be **rolled back**.

3.2 Adjacent capabilities

- **Releases** — consolidated per-fixVersion analysis with Excel export.
- **Revamp** — bulk Bedrock rewrite of all synced TestSigma cases into clean, stepped manual cases (reversible, cancellable).
- **Consolidate** — detect & merge near-duplicate cases.
- **TestSigma sync + autosync** — pull all cases, embed, incremental refresh on a per-workspace interval.
- **Observability** — decision/run/cost stats; full activity event log per run; approval audit trail.

3.3 Endpoints (grouped, prefix `/api/sdet-agent`)

Workspaces/provision · Runs (trigger/list/detail/preview/retry/events) · Decisions (bulk, edit, approve, reject, push, rollback, history) · Optimize · Releases (trigger/list/detail/push/export) · TestSigma sync & revamp · Consolidate · Autosync settings · Stats.

4. Pillar 3 — Test Library (TestManagement)

A complete test-management workspace. **28 `tm_*` tables**. Every entity is workspace-scoped, soft-deleted (`archived_at`), and audit-logged.

4.1 Repository

- **Suites** — *static* (explicit members + cases living in the suite's folders) or *dynamic* (filter-driven criteria). Health rollup (total/active/automated/coverage).
- **Folders** — recursive hierarchy with materialized path, optionally under a suite.
- **Test cases** — rich model: title, description, preconditions, priority, severity, type (manual/automated/hybrid), status (draft→in_review→active→deprecated), automation status, module/component, tags, owner/reviewer, **steps**, **test-data matrix**, labels, custom fields, **version history**, provenance (`source` : manual / ai_generated / testsigma_import / jira_derived / automation_derived / import_file), and an `automation_identity` to link automated results.
- **Steps** — each step is *Action / Test data / Expected result*; setup/test-data renders as readable label→value rows.
- **Labels** — colored, categorized, multi-assign.

4.2 Product templates

Per-workspace template tailors step labels and case custom fields for the product under test: **Generic, CodeScan, Vault, ARM, Guard, Salesforce.**

4.3 Execution

- **Plans** (release/sprint grouping) → **Cycles** (concrete run, environment-tagged) → **Cycle cases** (per-case `exec_status`, source, result, comment, defect ref, attachments).
- **Inline-checklist runner** — the cycle page is an inline checklist: a top case-stepper (pills + Prev/Next, `<` / `>`), each step marked `✓` / `✗` / `≡` in place, and the **case verdict rolls up automatically** from the marks. Result entry (actual result, comment, defect, attachments, Save/Skip) lives in a sticky right-hand panel; steps render full-height with no inner scroll.
- **Progress rollups** at cycle and plan level (executed %, pass rate, by-status).

4.4 Executions views

- **By folder (tree)** — Suite→Folder→Case with rolled-up pass/fail/blocked/skipped/untested counts and a segmented bar; result-filter pills prune the tree.
- **Recent activity** — flat latest-results feed with search.
- **Coverage gaps** — automated tests seen in RunView runs that have **no matching library case** (drift detector).

4.5 Automation mapping

RunView automated `TestResult`s link to TM cases by `automation_identity` (`classname::name`) → `name` → `case_id`. **Coverage report** (automated / matched / match-rate / by-status) and **cycle auto-sync** (set cycle results from the latest automated run).

4.6 Other sub-features

- **Requirements traceability** — requirements (manual/Jira) ↔ cases, coverage matrix (covered/uncovered/orphan).
- **Defects** — raise from a failed execution or manually; severity/status workflow; optional external (Jira) key.
- **Reviews & Approvals** — draft → in_review → active/rejected, with bulk actions and search.
- **Test Data sets** — reusable column/row matrices.
- **Import / Export** — CSV bulk (steps encoded inline).
- **Sync Center** — integration status hub (TestSigma / Jira / automation / defects).
- **AI Suggestions** — promote approved SDET-Agent decisions into native TM cases (idempotent).

4.7 Endpoints

~85 handlers under `/api/tm/workspaces/{ws}/...` grouped by: overview/settings · folders · test-cases (+labels/versions/clone) · labels · custom-fields · data-sets · import/export · integrations · reviews · defects · requirements · templates · mappings · suites · plans · cycles (+execution/automation-

sync/attachments) · automation (coverage/gaps/jobs/exec-mode/PRs) · executions (tree/recent) · ai-suggestions · audit.

4A. In-app automation (generate · execute · run)

RunView can author and run **Playwright** automation for a test case from the UI — no separate harness checkout required. The Flow A harness is vendored into the repo (`automation/`) and shipped as an opt-in Docker Compose service.

4A.1 Per-case lifecycle

On a case's detail page:

- **Generate Automation** — a **one-time** action (re-enabled only when the case's steps change, detected via a steps fingerprint). It drives the Claude Code CLI (AWS Bedrock) to author a spec, validate it against the live app, and self-heal up to N times; a real product failure is classified and surfaced rather than healed forever.
- **Execute the automated test case** — replays the existing spec (no Bedrock). Re-runnable until the steps change.
- A cycle's **"Run automated"** picker bulk-enqueues many cases at once (skips manual cases).

Each action enqueues a job in `tm_automation_jobs` (migration 0024). The queue is concurrency-safe (`claim_next` uses `FOR UPDATE SKIP LOCKED`). A job may only target a case **in its own workspace** (enforced at enqueue) — so results can only ever land in that workspace's runs.

4A.2 Execution mode — Docker vs CI (per workspace, superadmin)

Settings → Integrations → "Automation execution":

- **In-built Docker runner** (default) — a vendored Playwright container (`docker compose --profile automation up -d runner`) drains the queue **one case at a time** (single worker, headless); the rest stay queued. It logs in from the case's **Test Data** (inline values or an attached reusable data set), and for *generate* it pulls the **app-configured Bedrock** credentials at run time. Generated specs + learned knowledge persist on named volumes across restarts. `RUNNER_WS` accepts a comma-separated list to serve **multiple workspaces** (still one case at a time, round-robin). Stale/hung jobs self-heal (a reaper fails jobs stuck running past a TTL; the runner enforces per-job timeouts).
- **CI** — each job is dispatched to **Bitbucket Pipelines**, **AWS CodeBuild**, or a **generic webhook** (passing `WS/CASE_ID/KIND/JOB_ID`). The pipeline runs the single case via the shipped `ci-run.ts` and reports the result back (the harness ingests the run + posts job status). CI runs one container per case, fanning out across containers. Templates (`bitbucket-pipelines.yml` , `buildspec.yml`) ship in `automation/` .

4A.3 Results write-back & isolation

The harness ingests the run into **RunView Run Results** authenticated by **either** a per-project API key **or** the runner's bearer token (editor+), so one runner serves any workspace. Results map back to the case by the identity token and are scoped to the run's project — a workspace's automation only surfaces in that workspace.

4A.4 Approval-gated PR flow

A green spec posts a **pending PR request** (`tm_automation_pr_requests` , migration 0023) — nothing touches the repo yet. A reviewer approves it on the **Automation PRs** page (choosing the branch); only then does RunView commit the spec and open a Bitbucket PR. Per-workspace Bitbucket connection, configured in Settings.

4A.5 Automation queue page

A live (**Automation queue**, Test library nav) view of generate/execute jobs: summary tiles (queued/running/succeeded/failed, execution mode, last claim), status-filter tabs, paginated history, per-job log, **retry/cancel**, and a runner-offline hint when docker-mode has queued work but no recent claim.

5. Frontend architecture (SPA)

- **Stack** — React 19 + TypeScript (strict) + React Router 7, built by Vite → emitted into `backend/app/static` and bundled into the backend image.
 - **Shell** — `AppShell` with a collapsible icon rail (5 zones: Home, Results, Agent, Test Library, Admin), a top bar (workspace switcher, theme/density toggles, **? Tour** launcher, user menu), and a sticky secondary nav per zone.
 - **Interactive product tour** — a dependency-free guided walkthrough (`ProductTour`): a dimmed spotlight highlights each area while a bottom-right caption card steps through navigation, Dashboard, Runs/Tests/Monitor, Test Library, Cycles, Automation queue/PRs, SDET Agent and Admin (the admin step is role-gated). Auto-offered once on first visit; relaunchable anytime.
 - **State** — `MeContext` (user + role gating), `WorkspaceContext` (active project, persisted in `localStorage`), `api.ts` client (~200 endpoints, Bearer token from `localStorage.runview_token` , auto-logout on 401).
 - **Theming** — light/dark via `data-theme` , comfortable/compact density, persisted.
 - **Shared components** — `PageHeader` , `StatusBadge` , `Icon` , `FilterPanel` , `TmTablePanel` + `useClientTable` (client pagination), `TmSelect` , `WorkspaceSwitcher` (grouped command palette), `DocumentLayout` / `WorkbenchLayout` .
-

6. Deployment & operations

- **Image** — multi-stage Dockerfile: Node 22-alpine builds the SPA → Python 3.12-slim runtime, running as non-root `appuser` (UID 10001), `uvicorn app.main:app`.
- **Compose** — `postgres` (pgvector/pg17, internal network, not host-exposed) + `api` (exposed on `127.0.0.1:8080`), plus an opt-in `runner` service (profile `automation`) — the vendored Playwright + Claude Code harness that drains the automation queue, with named volumes for generated specs/knowledge/auth state. All services run with healthchecks, `no-new-privileges`, dropped capabilities, memory/PID limits.
- **Migrations** — Alembic chain in `backend/app/sdet/alembic` (currently head **0024**); auto-applied on startup via `bootstrap_safe()` when `SDET_AUTO_MIGRATE` is enabled (default). All TM migrations (0013–0024) are **additive** — including 0023 (automation PR requests) and 0024 (automation jobs).
- **Startup** (`main.py` lifespan) — `init_db()` → gated `seed_sample_data()` (no demo data on hosted) → SDET bootstrap (migrate + upsert workspaces) → background loops: retention (24h), run-timeout (30m), cache refresh, TestSigma autosync (15m), and N worker-queue consumers.
- **Config** — env-driven (DB URL, `SECRET_KEY`, `ADMIN_EMAIL/PASSWORD`, retention, `SDET_AUTO_MIGRATE`, `SDET_WORKERS`, Bedrock concurrency, optional `OIDC_*` / `SSO_*` for SSO). Integration secrets (Bedrock/TestSigma/Jira/Rovo) stored **encrypted at rest** in `AppSetting`.

7. External integrations

System	Purpose	Connection / Auth
AWS Bedrock	Claude LLM (analysis, revamp, consolidate, performance) + Titan embeddings	boto3, IAM or encrypted access keys, region-configured
TestSigma	Test-case management (sync/create/update/delete)	REST, Bearer token (encrypted)
Jira / Atlassian	Story context, dev-status PRs, defects, requirements import	REST, Basic (email+token) or Bearer (encrypted)
Rovo (optional)	Atlassian AI story enrichment	REST, Bearer (encrypted), best-effort
GitHub (optional)	PR changed-file counts	REST, Bearer (encrypted)
CI/CD systems	Push test results	<code>X-API-Key</code> per project (bcrypt-verified) or editor+ bearer token (any workspace)
Bitbucket Pipelines / AWS CodeBuild / webhook	CI execution backend for in-app automation (run a case, report back)	Pipelines REST · CodeBuild <code>start_build</code> (boto3) · POST webhook — per-workspace, configured in Settings
Bitbucket (repo)	Approval-gated push of generated specs + open PR	REST, Basic (email+token), per-workspace (encrypted)
OIDC IdP (SSO) (optional)	Single sign-on — Okta / Azure AD-Entra / Google / Auth0 / any OIDC	OIDC discovery, auth-code + PKCE, ID-token via JWKS

Note: a hard project rule forbids using ReportPortal's hosted API; no such reference exists in the SDET module.

8. Authentication & SSO

Two ways to obtain the app's session JWT; the rest of the app is unchanged regardless of which is used.

Password (always available). Email + bcrypt password → HS256 JWT (24h). Admin bootstrapped from env on first boot; login is rate-limited.

SSO / OIDC (optional, opt-in). Generic OpenID Connect — works with **Okta, Azure AD/Entra, Google Workspace, Auth0**, or any OIDC provider via issuer discovery. **Off by default**; with nothing configured the app behaves exactly as a password-only deployment.

- **Configured in-app by a superadmin — Settings → Access → "Single sign-on (SSO / OIDC)":** enter Issuer URL, Client ID, Client Secret (write-only), scopes, button label, redirect URL, and the SSO-only toggle, then tick **Enable**. Config is stored in the DB (`AppSetting`) with the **client secret encrypted at rest** (Fernet) and never returned by the API. SSO turns on only when *Enabled* and issuer/client-id/secret are all present. (Environment variables `OIDC_*` / `SSO_*` remain a fallback for ops who prefer file-based config.)
- **Flow:** login page "Single sign-on" button → IdP (authorization-code + **PKCE**) → callback validates the ID token against the provider's **JWKS** (issuer, audience, nonce) → issues the normal app JWT (handed to the SPA via URL fragment).
- **Account policy — link-existing-only:** the IdP identity is matched to an existing user **by email**; unknown emails are rejected. **SSO never creates accounts** — an admin must add the user first. (`auth_provider` , `sso_subject` , `sso_issuer` are recorded on first SSO sign-in.)
- **SSO-only mode (default when enabled):** password login is hidden for non-admins; a **superadmin password break-glass** always remains (so an IdP outage can't lock you out). Untick *SSO-only* to keep both visible.
- **Endpoints:** `GET /api/auth/sso/config` (public; drives the login UI) · `GET /api/auth/sso/login` (→ IdP) · `GET /api/auth/sso/callback` · `GET / PUT /api/settings/sso` (superadmin: read/update config).
- **Roles unchanged:** RBAC (`viewer/editor/superadmin`) is applied to the SSO-linked user exactly as for password users.

See **ARCHITECTURE.md** for the full component/data-flow diagram and **SECURITY_AUDIT.md** for the security posture.